1. Criar e preencher matriz dinamicamente

Escreva um programa que:

- leia dois inteiros n e m
- aloque dinamicamente uma matriz n x m
- preencha a matriz com valores de 1 até n*m
- imprima os elementos
- libere a memória alocada

2. Ler matriz e encontrar o maior elemento

Escreva um programa que:

- leia n e m
- aloque dinamicamente uma matriz n x m
- leia os valores do usuário
- determine e imprima o maior valor da matriz
- libere a memória corretamente

3. Aumentar o tamanho de um vetor com realloc

Escreva um programa que:

- aloque inicialmente um vetor para 5 inteiros
- leia 5 valores
- use realloc para aumentar o vetor para 10 posições
- leia os 5 novos valores
- imprima todos os 10 elementos
- libere a memória

4. Copiar vetor para outro vetor dinâmico

Escreva um programa que:

- leia um inteiro n
- aloque dinamicamente dois vetores de tamanho n
- leia os valores do primeiro vetor
- copie os valores para o segundo vetor
- imprima o segundo vetor
- libere toda a memória usada

Restrições:

- faça a cópia manualmente com laço
- não use memcpy

5. Remover um elemento de um vetor

Escreva um programa que:

- leia um inteiro n
- aloque dinamicamente um vetor de n inteiros
- leia os valores do vetor
- leia um valor x a ser removido
- remova a primeira ocorrência de x
- reduza o vetor com realloc
- imprima o vetor resultante

Observação:

- caso x não exista, o vetor deve permanecer inalterado

6. Inserir elemento em posição específica

Escreva um programa que:

- leia um inteiro n
- aloque dinamicamente um vetor de n inteiros
- leia os elementos
- leia um valor x e uma posição p
- aumente o vetor em 1 posição usando realloc
- insira x na posição p, deslocando os elementos à direita
- imprima o vetor final
- libere a memória

Considere posições válidas de 0 até n.

7. Concatenação de strings com alocação dinâmica

Escreva um programa que:

- leia duas strings
- aloque dinamicamente uma terceira string com tamanho suficiente para armazenar a concatenação das duas
- copie manualmente os caracteres
- adicione o terminador '\0'
- imprima a string resultante
- libere a memória

Restrições:

- não usar strcat
- não usar strcpy

8. Vetor Redimensionável

Escreva uma função que receba um ponteiro para um vetor de inteiros alocado dinamicamente e seu tamanho atual. A função deve ler novos números do usuário até que ele digite 0. Se o vetor ficar cheio, use realloc para dobrar o tamanho dele. Não esqueça de tratar o caso de falha no realloc.

9. Cópia de Strings Personalizada

Crie uma função **char* clonar_string(char* s)** que meça o tamanho da string s, aloque exatamente a quantidade de memória necessária (incluindo o caractere nulo \0) e copie o conteúdo. O programa principal deve liberar a memória após o uso.

10. Média de Alunos

Crie um programa que pergunte ao usuário quantos alunos existem na turma. Aloque dinamicamente um vetor de float para armazenar as notas. Após calcular a média, libere a memória.

11. Inversão de Vetor

Peça ao usuário um valor N. Aloque dinamicamente um vetor de inteiros de tamanho N. Preencha o vetor com dados do usuário e, em seguida, crie um segundo vetor dinâmico para armazenar os elementos do primeiro vetor em ordem inversa. Imprima o segundo vetor e libere ambos.

12. Concatenar Vetores

Crie uma função que receba dois vetores dinâmicos e seus tamanhos, e retorne um terceiro vetor (também dinâmico) que seja a junção dos dois.

13. Matriz dinâmica sem struct

Escreva um programa que:

- leia n e m
- aloque dinamicamente uma matriz de inteiros com n linhas e m colunas usando **int ****
- leia os valores da matriz
- imprima a soma de cada linha

- libere corretamente toda a memória

14. Matriz em bloco contínuo

Escreva um programa que:

- leia n e m
- aloque dinamicamente uma matriz usando um único bloco contínuo de memória
- leia os valores da matriz
- imprima a matriz
- calcule a soma de todos os elementos
- libere a memória

15. Dicionário de Palavras

Crie um programa que armazene uma lista de nomes (strings). O usuário diz quantos nomes quer digitar. Como cada nome tem um tamanho diferente, você deve alocar um **char **lista**, onde cada **lista[i]** deve ser alocado com o tamanho exato de cada nome lido (use malloc e strlen).

16. Matriz Dinâmica Clássica

Escreva um programa que aloque uma matriz de inteiros M x N usando um ponteiro para ponteiro (**int ****). O programa deve preencher a matriz com valores aleatórios, imprimir a transposta dela e liberar toda a memória (cada linha individualmente e depois o ponteiro principal).

17. Triângulo de Pascal Dinâmico

Solicite ao usuário um número de linhas L. Aloque uma estrutura de matriz onde cada linha i tenha exatamente i+1 elementos (uma matriz triangular). Calcule e armazene os valores do Triângulo de Pascal nela.

18. Remoção de Elementos

Implemente uma função que receba um vetor dinâmico de inteiros e remova todos os números negativos, utilizando realloc para diminuir o tamanho do vetor para o tamanho exato final após a remoção.

19. Cadastro dinâmico de notas com struct

Escreva um programa que:

- defina uma struct Aluno com:
  - nome do aluno
  - notas (vetor dinâmico)
- leia a quantidade de alunos n
- **declare um array de structs Aluno de tamanho n**
- para cada aluno:
  - leia o nome
  - leia a quantidade de notas
  - aloque dinamicamente as notas
  - leia as notas
- calcule e imprima:
  - a média de cada aluno
  - a média geral
- libere toda a memória ao final

20. Gerenciamento de Notas Multidisciplinares

Crie um programa que gerencie as notas de X disciplinas para um aluno. Primeiro, pergunte quantas disciplinas o aluno cursou (X). Aloque um array de ponteiros **float **boletim**. Para cada disciplina, pergunte quantas notas ela teve e aloque o espaço exato para aquelas notas naquela "linha" do boletim. Ao final, exiba a média de cada disciplina e a média geral do aluno, garantindo que toda a memória (cada disciplina e o array de disciplinas) seja liberada corretamente.